
name: ticket description: Atomic issue and task ticket creation skill. Deconstructs architectural specifications into bounded, independently executable work tickets.

Ticket — Atomic Task Deconstruction & Ticket Creation

Purpose & Protocol

The **Ticket** skill breaks down large architectural specs into self-contained, atomic work units. A single ticket must represent an isolated engineering change that can be implemented, tested, and reviewed by an autonomous agent in a single session without context loss.

The Ticket Template

Each generated ticket must follow this exact markdown template:

[TICKET-<ID>] <Short Descriptive Title>

1. Objective

One clear, unambiguous sentence describing the exact state change to be achieved.

2. Context & Parent Spec

- **Parent Specification:** `[[<Path to Spec>]]`
- **Subsystem:** `<target-repo>/<subfolder>`
- **Prerequisites:** List of tickets that must be completed prior to starting.

3. Scope of Work (Exact Files)

- `MODIFY:` Specific relative file paths to alter.
- `CREATE:` New files to create (with exact paths).

- `DO NOT TOUCH:` Files that must remain unmodified to prevent regressions.

4. Technical Constraints & Invariants

- Invariants from the parent spec that apply directly to this task.
- Memory, token, or dependency limitations.

5. Acceptance Test Criteria

- [] Unit tests pass via deterministic CLI command (`pytest ...` / `npm test ...`).
- [] Exit code 0 returned upon valid inputs; non-zero on malformed inputs.
- [] No undeclared imports or broken relative links introduced.

Ticket Sizing Heuristic

If a ticket touches more than 3 distinct files or requires more than 150 lines of new code, it is too large. Split it into sequential phase tickets (e.g., **TICKET-12A: Data Schema**, **TICKET-12B: Parser Engine**, **TICKET-12C: CLI Wrapper**).